Reinforcement Learning for Dynamic Food Delivery Dispatch

Project Plan and Implementation Specification

Anshit Agarwal Namya Dhingra

October 23, 2026

1 Project Overview

The project aims to formulate and solve a real-world-inspired food delivery dispatch problem using Reinforcement Learning (RL). The system simulates a food-delivery platform where restaurants receive orders, orders take time to prepare, and delivery partners must be assigned to customers efficiently.

The central decision-making problem is:

> Given the current orders, restaurants, delivery partners, preparation status, locations, and delivery conditions, which delivery partner should receive which order, and when should the system wait before making an assignment?

The project will compare five different decision-making strategies under the exact same simulated environment:

1. Baseline 1 — Nearest Driver
2. Baseline 2 — Oldest/Most Urgent Order
3. Baseline 3 — Minimum Estimated Delivery Time
4. DQN
5. PPO

The RL-based approaches will learn their decision-making policies from interaction with the simulated environment, while the heuristic approaches will serve as baselines for comparison.

The project is intentionally designed as a manageable simulator rather than an attempt to reproduce the proprietary systems of real food-delivery companies.

2 Why Reinforcement Learning?

Food delivery dispatch is naturally sequential. At every point in time:

- new orders can arrive,
- existing orders can become ready,
- drivers can become available,
- drivers can finish deliveries,
- preparation times can change,
- travel times can differ,
- waiting for additional orders can sometimes enable batching,
- assigning a driver now can affect future decisions.

Therefore, a decision that looks optimal at the current moment may not be optimal over the entire episode.

For example, suppose Order O1 is ready now and Order O2 will be ready in two minutes. A driver can deliver O1 immediately, or the system can wait and potentially batch O1 and O2. This sequential trade-off makes the problem suitable for RL.

3 RL Formulation

The problem will be formulated as a Markov Decision Process:

$$M = (S, A, P, R, \gamma)$$

where:

- S = state space,
- A = action space,
- P = transition dynamics,
- R = reward function,
- γ = discount factor.

The RL agent acts as the central dispatch decision-maker. The environment is the food-delivery simulator.

At every decision point:

$$s_t \to a_t \to r_t \to s_{t+1}.$$

The agent observes the current state, chooses an action, receives a reward, and the simulator transitions to the next state.

4 Simulation Environment

The environment will represent a simplified city using a 2D coordinate/grid system.

The environment contains:

- Restaurants
- Customers
- Delivery partners
- Orders
- Travel distances/times
- Food preparation times
- Delivery deadlines
- Simulation clock

The environment will operate using discrete simulated time steps, such as one simulated minute per step.

4.1 Fixed Simulation Duration

A major experimental constraint is that **every simulation episode will have exactly 3 simulated minutes of duration**:

$$T_{\text{episode}} = 3 \text{ simulated minutes.}$$

This duration is fixed regardless of the selected configuration or operating conditions.

The simulation will always terminate when the 3-minute simulated horizon is reached. The fixed duration applies equally to:

- Baseline 1,
- Baseline 2,
- Baseline 3,
- DQN,
- PPO.

This is important for fair comparison because an algorithm must not receive additional simulation time simply because of the configuration or strategy being used.

The 3-minute duration refers to **simulated time**, not real wall-clock execution time. A 3-minute episode may therefore execute much faster than three real minutes.

The GUI will **not contain a simulation-duration slider**. Simulation duration is a fixed experimental parameter rather than a user-configurable parameter.

5 Order Lifecycle

Every order will move through a sequence of states:

$$Created \rightarrow Preparing \rightarrow Ready \rightarrow Assigned \rightarrow Picked\ Up \rightarrow Delivered.$$

An order may therefore spend time:

- waiting for preparation,
- waiting for driver assignment,
- travelling to the restaurant,
- waiting for pickup,
- travelling to the customer.

The simulator will track these times because they directly affect the reward and evaluation metrics.

6 Delivery Partner Model

Each delivery partner will have:

- Current location
- Availability status
- Current assigned order(s)
- Current route/status
- Travel progress

Possible statuses include:

- `AVAILABLE`
- `TO_RESTAURANT`
- `TO_CUSTOMER`
- `IDLE`

The simulator will update driver positions as simulated time progresses.

7 State Space

The state should contain enough information for the RL agent to make an informed dispatch decision.

For each delivery partner:

$$s_i^{driver} = [x_i, y_i, available_i, load_i, status_i].$$

For each order:

$$s_j^{order} = [x_j^r, y_j^r, x_j^c, y_j^c, age_j, prep_j, ready_j, deadline_j].$$

Global information may include:

- Current simulation time
- Number of pending orders
- Number of available drivers
- Number of orders being prepared

- Current demand level

Because neural networks require fixed-size inputs, the implementation will use a fixed maximum number of active drivers/orders and padding where necessary.

The exact dimensionality of the final observation vector will be finalized during environment implementation.

8 Action Space

The primary action is a driver-order assignment:

$$a = assign(d_i, o_j).$$

The agent may also have:

$$a = wait.$$

Therefore, the action space contains possible driver-order assignments plus a waiting action.

Invalid assignments will be masked or rejected.

Action masking may be used to prevent the agent from selecting obviously impossible actions, such as assigning an order to an unavailable driver.

The same conceptual action space will be used for both DQN and PPO.

9 Waiting and Order Batching

Waiting is an important part of the problem.

For example:

- O1 is ready now.
- O2 will become ready shortly.
- A driver can deliver O1 immediately.
- Alternatively, the system can wait and potentially batch the orders.

The agent therefore needs to learn whether waiting is beneficial or harmful.

Batching can reduce:

- total travel,
- driver utilization,
- delivery cost.

However, excessive waiting can increase:

- customer waiting time,
- late deliveries,
- order age.

Therefore, the reward function must balance these competing objectives.

The initial implementation should prioritize getting single-order dispatch working correctly. Batching can then be incorporated as an extension once the core environment is stable.

10 Reward Function

The reward will combine several objectives.

A general formulation is:

$$R_t = w_1 D_t - w_2 W_t - w_3 L_t - w_4 T_t + w_5 B_t - w_6 I_t,$$

where:

- D_t = successful deliveries,

- W_t = customer waiting penalty,
- L_t = late-delivery penalty,
- T_t = travel cost,
- B_t = useful batching benefit,
- I_t = driver idle penalty,
- $w_1, \ldots, w_6$ = reward weights.

The final weights will be determined during experimentation and then kept fixed during evaluation.

The reward must not allow the agent to optimize only one metric, such as minimizing distance while producing poor delivery times.

11 Baseline Strategies

Three heuristic strategies will be implemented.

11.1 Baseline 1 — Nearest Driver

Assign an order to the closest available driver.

This provides a simple spatial heuristic.

11.2 Baseline 2 — Oldest/Most Urgent Order

Prioritize the order that has been waiting the longest or has the highest urgency, and assign a feasible driver.

This prioritizes customer waiting time and delivery urgency.

11.3 Baseline 3 — Minimum Estimated Delivery Time

Evaluate feasible driver-order pairs and select the assignment with the smallest estimated delivery time.

This provides a stronger heuristic baseline because it directly considers expected delivery performance.

These methods are **baselines only**. The core project remains an RL-based solution.

12 Deep Q-Network (DQN)

DQN will be implemented by the project partner.

DQN approximates the action-value function:

$$Q(s, a).$$

A neural network receives the current state and produces a Q-value for each possible action.

The agent selects:

$$a^* = \arg\max_a Q(s, a)$$

during exploitation.

The DQN implementation will include:

- Neural-network Q-function
- Experience replay
- Target network
- Epsilon-greedy exploration
- Mini-batch training

The partner will be responsible for the DQN training and evaluation pipeline.

13 Proximal Policy Optimization (PPO)

PPO will be implemented by Anshit.

Instead of directly estimating Q-values, PPO learns a policy:

$$\pi_\theta(a|s),$$

which represents the probability of selecting an action given the current state.

PPO uses a clipped objective to prevent excessively large policy updates and provide stable training.

The PPO implementation will include:

- Policy network
- Value network
- Rollout collection
- Advantage estimation
- Clipped policy objective
- Policy/value updates

14 Complete RL Loop

The overall RL process is:

$$\boxed{State \rightarrow Agent \rightarrow Action \rightarrow Environment \rightarrow Reward \rightarrow Next\ State}$$

For every decision:

1. Environment provides the current state.
2. RL agent selects an action.
3. Action is executed by the simulator.
4. Environment updates drivers, orders and time.
5. Reward is calculated.
6. Next state is generated.
7. The process continues until the fixed 3-minute simulation horizon is reached.

At the end of the episode, the accumulated performance is recorded.

15 Episodes

One episode represents **one complete 3-minute simulated delivery scenario**.

The episode begins with an initial configuration containing:

- Restaurant locations
- Driver locations
- Customer/order generation parameters
- Preparation times
- Travel conditions

Orders then arrive dynamically during the 3-minute simulation.

The episode terminates automatically after exactly 3 simulated minutes.

Multiple episodes will be generated during training so that the RL agents experience different delivery scenarios.

16 Training Procedure

DQN and PPO will be trained by repeatedly interacting with the simulator.

A simplified training loop is:

```
Initialize environment
Initialize RL agent

For each training episode:
    Reset environment
    Run simulation for exactly 3 simulated minutes

    While simulation time < 3 minutes:
        Observe state
        Select action
        Execute action
        Receive reward
        Observe next state
        Store/update learning information

    Update RL agent
```

The environment used during training will be the same environment used for evaluation, although evaluation scenarios/seeds will be kept separate from training data.

17 Fair Comparison Protocol

Fair comparison is one of the most important parts of the project.

All five strategies will operate on the **same environment** and under identical experimental conditions.

For a given experiment, all strategies will receive the same:

- Restaurant locations
- Customer locations
- Initial driver positions
- Order arrival sequence
- Food preparation times
- Travel/traffic conditions
- Simulation configuration
- Random seed
- **3-minute simulation horizon**

Thus, the only major difference is the decision-making strategy.

The fixed 3-minute horizon ensures that no algorithm gets an advantage from being evaluated for a longer period.

18 Evaluation Metrics

The following metrics will be recorded.

18.1 Average Delivery Time

$$Average\ Delivery\ Time = \frac{\sum Delivery\ Times}{Number\ of\ Deliveries}.$$

18.2 Average Customer Waiting Time

Measures how long customers wait before receiving their orders.

18.3 On-Time Delivery Percentage

$$OnTime\% = \frac{On-Time\ Deliveries}{Total\ Deliveries} \times 100.$$

18.4 Late Delivery Percentage

$$Late\% = \frac{Late\ Deliveries}{Total\ Deliveries} \times 100.$$

18.5 Total Travel Distance

Total distance travelled by all delivery partners.

18.6 Driver Utilization

Measures how effectively the available driver fleet is being used.

18.7 Successful Batching Count

Number of useful multi-order delivery decisions.

18.8 Episode Reward

The cumulative discounted reward is:

$$G = \sum_t \gamma^t R_t.$$

All metrics will be calculated over the same 3-minute episode horizon.

19 Experimental Scenarios

The initial illustrative scenarios are:

Table 1: Initial Experimental Configurations

Scenario	**Restaurants**	**Drivers**	**Orders/min**
Low Demand	5	8	2
Medium Demand	5	8	5
High Demand	5	8	8
Driver Shortage	5	4	5
Driver Surplus	5	12	5

Every individual experiment will still run for exactly **3 simulated minutes**.

The scenarios are designed to test how the algorithms behave under different levels of demand and resource availability.

20 Game-Like Simulation Interface

The project will include a simple game-like visualization.

The configuration screen will allow the user to modify parameters such as:

- Number of restaurants
- Number of delivery partners
- Order arrival rate
- Traffic/travel conditions
- Batching settings, if applicable

Simulation duration will not be configurable because it is fixed at 3 simulated minutes.

The interface will provide an algorithm-selection option:

- Baseline 1

- Baseline 2
- Baseline 3
- DQN
- PPO
- Run All

The simulation map will display:

- Restaurants
- Customers
- Drivers
- Orders
- Driver routes
- Delivered orders

At the end of the episode, the interface can display a leaderboard comparing the strategies.

21 Software Architecture

The system will consist of five major layers:

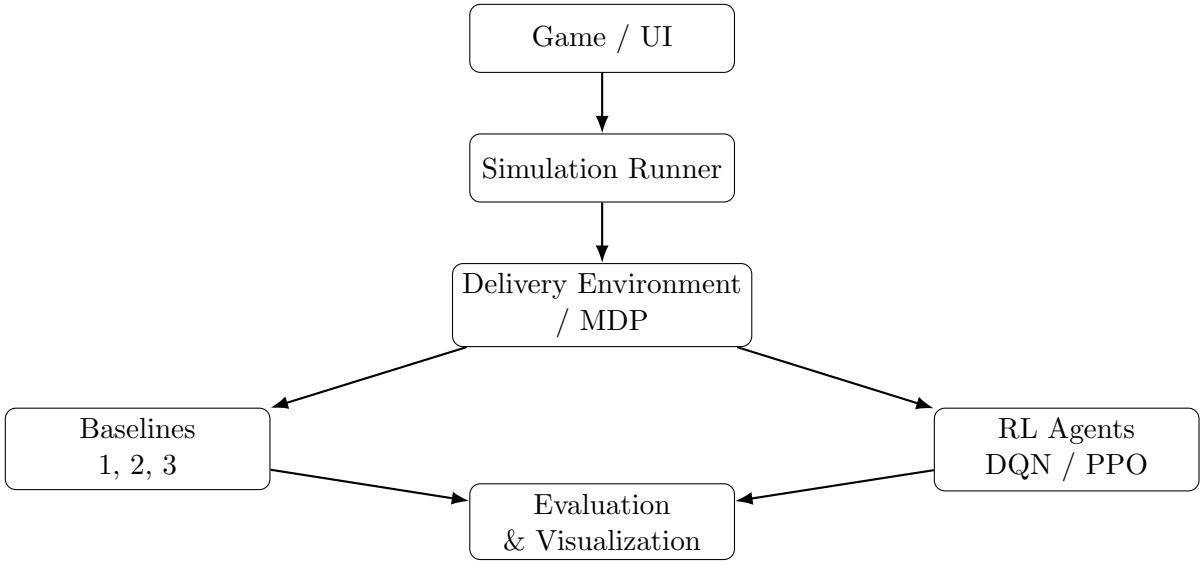


Figure 1: Overall Software Architecture

22 Proposed File Structure

```
project/
 README.md
 requirements.txt

 environment/
  delivery_env.py
  order.py
  driver.py
  restaurant.py
  city.py
  config.py

 baselines/
  baseline_1.py
  baseline_2.py
```

```
 baseline_3.py

rl/
 dqn/
  model.py
  train.py
  evaluate.py

 ppo/
 model.py
 train.py
 evaluate.py

simulation/
 simulator.py
 runner.py

evaluation/
 metrics.py
 comparison.py
 visualization.py

models/
 dqn/
 ppo/

results/
 raw/
 tables/
 plots/

ui/
 game.py

report/
    report.tex
```

23 Module Responsibilities

23.1 environment/

Contains the core MDP and all simulation entities.

23.2 baselines/

Contains the three heuristic decision-making strategies.

23.3 rl/dqn/

Contains the DQN model, training pipeline and evaluation code.

23.4 rl/ppo/

Contains the PPO model, training pipeline and evaluation code.

23.5 simulation/

Controls episode execution and simulation progression. The simulation engine will enforce the fixed 3-minute episode duration.

23.6 `evaluation/`

Calculates metrics and generates comparison plots/tables.

23.7 `ui/`

Contains the game-like visualization and configuration interface.

23.8 `models/`

Stores trained DQN and PPO models.

23.9 `results/`

Stores raw results, tables and generated plots.

24 Team Division

24.1 Anshit

Responsible for:

- Baseline 1
- Baseline 2
- PPO
- Initial game configuration
- UI sliders/configuration interface
- PPO integration
- Experimental integration

24.2 Namya

Responsible for:

- Baseline 3
- DQN
- DQN training pipeline
- DQN integration

24.3 Shared Responsibilities

Both members will work together on:

- Common environment
- Simulation engine
- MDP formulation
- State design
- Action design
- Reward design
- Evaluation
- Metrics
- Visualization
- Experiments
- README
- GitHub integration
- Testing
- Final report

The two RL implementations must use the same environment and evaluation framework.

25 Implementation Order

The project should be implemented in the following order.

25.1 Phase 1 — Environment

Build:

- Restaurants
- Drivers
- Orders
- Customers
- Time progression
- Travel
- Order lifecycle

First verify that the simulator correctly runs for exactly **3 simulated minutes**.

25.2 Phase 2 — MDP

Implement:

- State representation
- Action space
- Reward function
- Reset/step mechanism

25.3 Phase 3 — Baselines

Implement and test:

- Nearest Driver
- Oldest/Most Urgent Order
- Minimum Estimated Delivery Time

25.4 Phase 4 — RL

Implement:

- DQN
- PPO

25.5 Phase 5 — Evaluation

Run all five strategies under identical scenarios and seeds.

25.6 Phase 6 — GUI

Add the game-like visualization and configuration interface.

25.7 Phase 7 — Final Experiments

Run experiments across different demand/resource configurations, with every episode fixed to 3 simulated minutes.

26 Expected Results and Analysis

The primary objective is not necessarily for one algorithm to dominate every metric. Instead, the experiments should reveal the trade-offs between strategies.

For example:

- A nearest-driver strategy may reduce immediate travel distance.
- An urgency-based strategy may reduce customer waiting.
- Minimum-ETA may provide strong heuristic performance.
- DQN may learn useful action-value patterns.
- PPO may learn more strategic sequential behavior.
- RL agents may perform particularly well when demand and resource availability create complex trade-offs.

The final analysis will compare the algorithms using multiple metrics rather than relying only on reward.

27 Graphs and Visualizations

Potential final plots include:

1. Average Delivery Time vs Algorithm
2. Average Waiting Time vs Algorithm
3. On-Time Delivery % vs Algorithm
4. Total Travel Distance vs Algorithm
5. Driver Utilization vs Algorithm
6. Episode Reward vs Algorithm
7. Performance under Low/Medium/High Demand
8. Performance under Driver Shortage/Surplus
9. Training Reward Curve for DQN
10. Training Reward Curve for PPO

All evaluation episodes will use the same fixed 3-minute horizon.

28 Important Design Constraints

The following constraints should remain fixed throughout the project:

1. The project must contain a genuine RL component.
2. Heuristic algorithms are used only as baselines.
3. DQN and PPO must use the common delivery environment.
4. All five strategies must be evaluated under identical conditions.
5. Invalid actions must be handled appropriately.
6. Reward design must represent multiple delivery objectives.
7. Random seeds should be controlled for fair comparisons.
8. Training and evaluation scenarios should be separated where possible.
9. The simulator should remain sufficiently small and interpretable.
10. **Every simulation episode must run for exactly 3 simulated minutes.**
11. **Simulation duration must not depend on demand, number of drivers, number of restaurants, or algorithm.**
12. **Simulation duration must not be exposed as a GUI configuration slider.**

29 Final End-to-End Pipeline

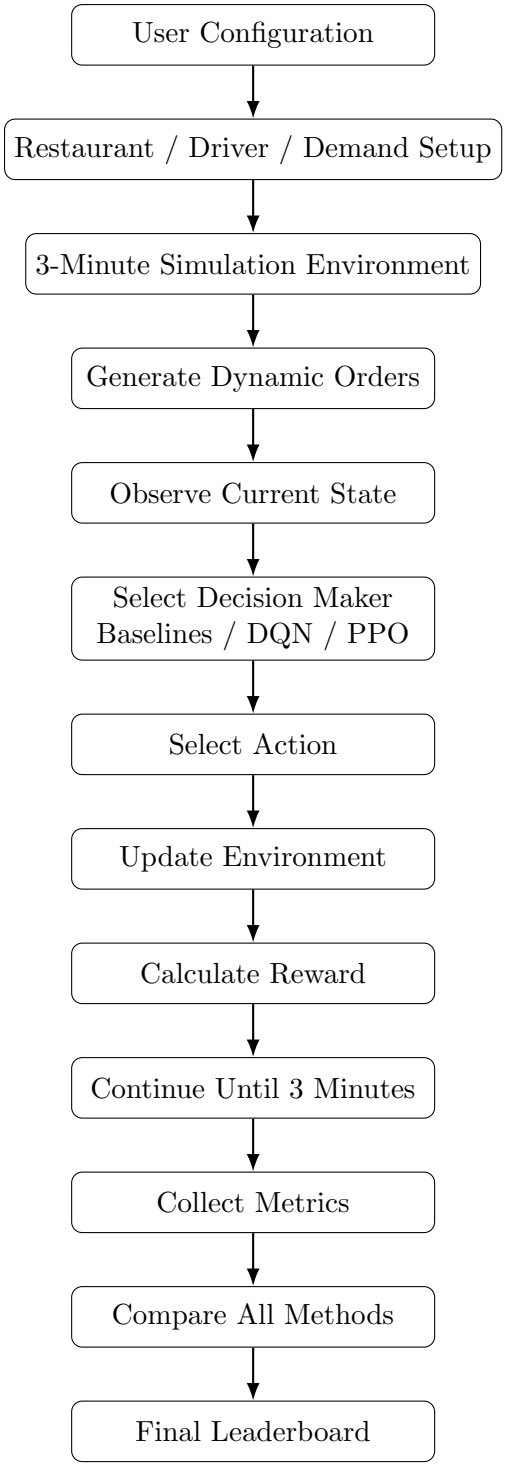


Figure 2: End-to-End Project Pipeline

30 Final Architecture

The conceptual architecture is:

Food Delivery Problem → MDP Formulation → 3-Minute Simulator → Baselines → DQN + PPO → Experiment

The simulator provides a controlled environment in which different decision-making strategies can be tested under identical conditions.

Each episode represents a complete **3-minute simulated delivery period**, and the same fixed horizon is used for every algorithm and every experimental configuration.

31 Final Objective

The final project should demonstrate the complete application of Reinforcement Learning to a realistic sequential decision-making problem:

Real-world-inspired problem + MDP + Simulation + Heuristic baselines + DQN + PPO + Experiments + Game-li

The project should ultimately answer:

> **Can a learned RL dispatch policy make better sequential food-delivery assignment decisions than simple heuristic strategies when all methods operate in the same 3-minute simulated environment?**

The project is therefore not simply an implementation of DQN or PPO. The core contribution is the complete formulation, simulation, learning, and experimental comparison of a dynamic food-delivery dispatch problem.